13
Design an ATM System

In this chapter, we will explore the object-oriented design of an ATM system. The primary purpose of an ATM is to establish banking tasks for users, allowing them to check balances, withdraw funds, and transfer funds. The design aims to make these transactions straightforward while ensuring strong security measures. Key components, such as the ATM machine, bank accounts, hardware interfaces, and transaction logs,

Automated Teller Machine (ATM)

Let's gather the specific requirements through an extended interview scenario.

Requirements Gathering

Interviewer: What type of system do you want to design?

Candidate: We are designing an ATM system for a diverse clientele to manage your banking needs. You insert your card, enter your PIN, and then choose from various options such as checking balances, withdrawing cash, or depositing funds. Within sessions, the system verifies your credentials and processes your request. Behind the scenes, the ATM coordinates with the bank, manages account balances, and ensures secure transactions. The system also keeps a log of all transactions for your records. Our design focuses on ATM operations rather than internal bank operations, ensuring smooth and secure transactions.

Requirements clarification

The first step in designing the ATM system is to understand precisely what the interviewer wants you to design. Here is an example of how a conversation between a candidate and the interviewer might unfold:

Interviewer: What functions does the ATM support?

Candidate: The ATM supports key functions such as checking balances, cash withdrawals, and fund transfers. Users can also view their transaction history, ensuring they have a clear overview of their financial activities. The system also allows users to change their PIN for security and to exit, ending the session at any time. Does this flow align with your vision, or should I adjust any features?

Interviewer: Yes, that sounds good. The system should support PIN entry, cash withdrawals, balance inquiries, fund transfers, and transaction logs. Is there anything else you'd like to include?

Candidate: No, that covers the essentials. We can focus on these functions for now.

Interviewer: Great. Let's proceed with the design. Please ensure the system is secure and user-friendly.

Candidate: Understood. We'll prioritize security and user-friendliness in the design. Let's start by outlining the main components and their interactions.

Interviewer: Sounds good. Please include a Card Reader, Keypad, Screen, Cash Dispenser, Deposit Slot, and an optional Printer for receipts.

Candidate: Noted. The system should also support multiple accounts per customer and ensure data consistency across transactions.

Interviewer: Exactly. We want to avoid discrepancies in account balances.

Candidate: Understood. We'll use database transactions and locking mechanisms to maintain data integrity.

Interviewer: Let's focus on Withdraw and Deposit for simplicity. Transfers are out of scope for now. Ensure withdrawals validate sufficient funds before dispensing cash.

Candidate: Sounds solid. I suggest the ATM displays clear messages for issues like insufficient funds, invalid PINs, or system errors.

Interviewer: Agreed. Let's begin with a class diagram.

Requirements

Based on the requirements gathering dialogue, the following functional requirements are identified for the ATM system:

- **Insert Card:** The customer initiates a session by inserting their card into the ATM.
- **Enter PIN:** Within a single session, the customer can enter their PIN for authentication.
- **Select Transaction:** The customer chooses a transaction type, such as withdrawing cash or depositing funds, from the menu.
- **Withdraw Cash:** The customer can withdraw cash from their account, ensuring sufficient funds are available.
- **Withdraw Cash:** The system validates the customer's PIN and authorizes the transaction.
- **Deposit Funds:** The customer deposits cash into the ATM.
- **Check Balance:** The customer can view their account balance.
- **Transfer Funds:** The customer can transfer funds between accounts.
- **View Transaction History:** The customer can view a log of all transactions for their account.
- **Eject Card:** The ATM returns the customer's card after the session ends.

The ATM machine interacts with various hardware components, including a card reader, keypad, screen, cash dispenser, deposit slot, and optional printer. The system also maintains a log of all transactions for security and audit purposes.

Use Case Diagram

In the ATM system, a user (the customer) interacts with the system to perform banking tasks, including inserting their card, entering their PIN, and selecting transactions such as withdrawals, account selection, transaction processing, and viewing transaction logs.

Below is the use case diagram of the ATM system:

Use Case Diagram of ATM

The **Customer** actor has the following main use cases:

- **Insert Card:** The customer initiates a session by inserting their card into the ATM.
- **Enter PIN:** Within a single session, the customer can enter their PIN for authentication.
- **Select Transaction:** The customer chooses a transaction type, such as withdrawing cash or depositing funds, from the menu.
- **Withdraw Cash:** The customer can withdraw cash from their account, ensuring sufficient funds are available.
- **Withdraw Cash:** The system validates the customer's PIN and authorizes the transaction.
- **Deposit Funds:** The customer deposits cash into the ATM.
- **Check Balance:** The customer can view their account balance.
- **Transfer Funds:** The customer can transfer funds between accounts.
- **View Transaction History:** The customer can view a log of all transactions for their account.
- **Eject Card:** The ATM returns the customer's card after the session ends.

The ATM machine interacts with various hardware components, including a card reader, keypad, screen, cash dispenser, deposit slot, and optional printer. The system also maintains a log of all transactions for security and audit purposes.

Identify Core Objects

To design the ATM system effectively, we identify core objects that encapsulate distinct responsibilities, aligning with the functional requirements and use cases. These objects manage the system's key entities and interactions, ensuring clear separation of concerns and maintainability.

Bank: Stores and manages accounts.

Account: Represents a customer's bank account. Manages a customer's account details, including balance, account number, and associated PIN. Handles fund transfers and balance updates.

Card: Represents a customer's bank card. Stores card details, such as the card number, and links to the associated account.

CashDispenser: Manages the ATM's cash inventory. Tracks available cash and ensures sufficient funds are available for withdrawals.

CardReader: Reads card data when a card is inserted. Interacts with the card to extract account information.

Keypad: Captures user input, such as PIN entry and transaction selections.

Screen: Displays messages and prompts to guide the user.

DepositSlot: Manages the deposit of cash into the ATM.

BankAdapter: Acts as an interface between the ATM and the bank's backend systems, facilitating communication for account verification, balance updates, and transaction processing.

TransactionLog: Records all transactions for auditing and security purposes. Stores details of each transaction, including type, amount, and timestamp.

Transaction: Represents a generic transaction. Serves as a base class for specific transactions, such as withdrawals and deposits.

Withdrawal: Extends the Transaction class. Manages the logic for cash withdrawals, including balance checks and cash dispensing.

Deposit: Extends the Transaction class. Manages the logic for cash deposits, including updating account balances.

BalanceInquiry: Extends the Transaction class. Retrieves and displays account balance information.

TransferFunds: Extends the Transaction class. Manages the logic for transferring funds between accounts.

ATMSystem: The central system that integrates all components, coordinating interactions between various objects to ensure a cohesive workflow.

Design Class Diagram

With the core objects and their roles defined, we now design their attributes, methods, and relationships to construct a modular and extensible ATM system that meets the specified requirements.

Account

This class represents a customer's bank account in the ATM system, encapsulating the essential data and operations associated with a bank account, such as balance management and PIN verification.

The Account class manages account information, including the account number, balance, and PIN. It provides methods for checking the balance, updating the balance, and verifying the entered PIN. The class ensures that all operations are secure and that the account data is consistent and accurate.

Account class in ATM System

Attributes:

- accountNumber: int
- pin: int
- balance: double

Methods:

- getBalance(): double
- setBalance(double b)
- getAccountNumber(): int
- verifyPin(int pin): boolean

Important discussion: Rather than maintaining a transaction log for auditability and declaring the balance field as a double to deal rounding, we simplify the system by directly updating the account balance during each transaction. In a detailed implementation, transaction logs would be essential for auditability and reconciliation.

BankAdapter

A bank's internal design systems are usually self-contained and don't use standards or web services API calls. The Bank class is designed to be a separate component that handles the main data and operations needed for the ATM to work.

The Bank class stores account details and provides methods to verify PIN numbers, retrieve account balances, and process withdrawals and deposits. The class encapsulates the transaction logic, ensuring data consistency and security.

Bank class in ATM System

Attributes:

- bank: additionalBankObjects, accountNumber, accountType, balance
- authToken: String
- sessionID: String
- isAuthenticated: boolean
- isSessionActive: boolean
- isTransactionInProgress: boolean

Methods:

- authenticateSession(int accountNumber, int pin, String bankID): boolean
- updateBalance(int accountNumber, int newBalance, int pin): boolean
- withdrawFromAccount(int accountNumber, int amount, int pin): boolean
- depositToAccount(int accountNumber, int amount, int pin): boolean
- transferFunds(int fromAccount, int toAccount, int amount, int pin): boolean
- endSession()
- startTransaction()
- commitTransaction()
- rollbackTransaction()
- logTransaction(String transactionDetails)
- setSessionTimeout(int timeoutMinutes)
- extendSession()
- setAuthToken(String token)
- setSessionID(String sessionID)

Transaction

The purpose of designing the Transaction class is to provide a unified framework for handling financial operations in the ATM system, enabling the system to process various transactions such as withdrawals, deposits, and balance inquiries.

The Transaction class serves as a base class for all transactions, ensuring that each transaction is organized by a transaction type and has attributes and methods for managing transaction details. The class includes methods for processing transactions, updating the account balance, and logging transactions. Derived classes like Withdrawal and Deposit inherit from the Transaction class.

Transaction Interface and concrete classes

ATMState

The purpose of designing the ATMState interface is to provide a framework for handling the distinct stages of the ATM's interaction process, ensuring the ATMSystem can manage each step effectively.

The ATMState Interface defines the operations for each stage of the ATM's flow, such as card insertion, PIN entry, transaction selection, and cash handling. Concrete state classes like IdleState, PinEntryState, TransactionSelectionState, WithdrawalProcessingState, and CashDispensingState implement these operations, ensuring clear separation of stage-specific logic.

The following diagram shows the class hierarchy of ATMState and its concrete implementations:

ATMState Interface and concrete classes

The following state transition diagram visualizes this design:

State transition diagram

- IdleState: Awaits card insertion, transitions to PinEntryState on valid card.
- PinEntryState: Processes PIN entry, transitions to TransactionSelectionState on valid PIN.
- TransactionSelectionState: Processes transaction selection, transitions to WithdrawalAmountEntered or BalanceInquiry.
- WithdrawalAmountEnteredState: Processes withdrawal amount entry, executes transaction, returns to TransactionSelectionState or IdleState.
- CashDispensingState: Processes dispensed amount, executes transaction, returns to TransactionSelectionState or IdleState.

For further reading, see the State pattern and the concrete state classes in the folder **Sources/State** in Xcode.

Hardware component interfaces

The purpose of designing the hardware component interfaces is to provide a set of interfaces that handle user interactions and physical operations within the ATM system, ensuring the ATMSystem can operate independently of specific hardware implementations.

The hardware component interfaces including CardReaderInterface, Keypad, CashDispenser, and DepositSlot, define the operations needed for interacting with the ATM's hardware components. These interfaces include methods for reading card data, processing keypad inputs, dispensing cash, and collecting deposits. They enable the ATMSystem to perform these tasks through well-defined interfaces, ensuring flexibility and ease of maintenance for hardware integrations.

CardProcessor

The CardProcessor interface provides the ATMSystem to read a card during interface and release it after the user's session, managing card inserted operations for starting and ending transactions.

Keypad

The Keypad interface captures user inputs such as PIN, transaction choices, and amounts.

DepositSlot

The DepositSlot interface collects the deposited amount during a deposit transaction, enabling the ATMSystem to handle cash deposits securely.

CashDispenser

The CashDispenser interface delivers the requested cash to the user during a withdrawal transaction.

Display

The Display interface shows messages and prompts to guide the user.

The following diagram shows the hardware component interfaces and their relationship with the ATMSystem:

Hardware component interfaces

ATMMachine

The ATMMachine class acts as a facade for the ATM system, encapsulating user interactions, coordinating hardware components while relying on a State instance to access transaction data and process transactions. It uses the ATMState class to manage the transition of state objects and ensure that each state operates in the correct sequence from card insertion to cash dispensing.

Below is the representation of the ATMMachine class and its relationships:

ATMMachine class in ATM System

Design Patterns

The State pattern effectively manages state transitions in the ATM system, ensuring clear separation of concerns and maintainability. Each state encapsulates its specific behaviors, simplifying the system's overall structure and enhancing its adaptability to new requirements. This pattern ensures that the ATM system can easily accommodate new states or modifications to existing states without affecting the overall system architecture.

Complete Class Diagram

Take a moment to review the complete class diagram, which illustrates the relationships between classes. The detailed methods and attributes are omitted to make the diagram more readable.

Automated Teller Machine (ATM) System

Code - ATM System

In this section, we will implement the core functionalities of the ATM System, focusing on key areas such as defining different account types and transactions, managing accounts and banking operations, handling user interactions through hardware components, and coordinating transactions using various states.

Account

The Account class represents a bank account in our ATM system, encapsulating core banking attributes like balance, account number, and details, and account type. It maintains security through PIN hashing and is designed with a mix of immutable properties for critical data and mutable properties for balance updates. The class includes methods for PIN verification, balance updates, secure access control, and managing transaction histories.

Third-party APIs, like AccountType class defines the type of account a customer can hold, currently supporting CHECKING and SAVING. The Currency class manages currency-specific operations, ensuring proper formatting and validation for various currencies.

Below is the core implementation of the Account class and AccountType enum:

```
// Represents a bank account with balance, card details, and PIN security
// Core banking entity for ATM transactions
private BigDecimal balance;
private String accountNumber;
private String cardNumber;
private int pinHashCode;
private AccountType accountType;

// Creates a new account with initial zero balance and hashed PIN
public Account(
    String accountNumber,
    String cardNumber,
    int pin,
    AccountType accountType) {
    this.accountNumber = accountNumber;
    this.cardNumber = cardNumber;
    this.pinHashCode = pin.hashCode();
    this.accountType = accountType;
    this.balance = BigDecimal.ZERO;
}

// Updates account balance by adding the specified amount
// Used for deposits and successful withdrawals
public synchronized void updateBalance(BigDecimal amount) {
    balance = balance.add(amount);
}

// Gets current balance for display
public synchronized BigDecimal getBalance() {
    return balance;
}

// Verifies the type of PIN entered
// Security check before allowing access
public boolean verifyPin(int pin) {
    return pin.hashCode() == pinHashCode;
}

// Getters for various account details
// Read-only access to account information
public String getAccountNumber() {
    return accountNumber;
}

public String getCardNumber() {
    return cardNumber;
}

public AccountType getAccountType() {
    return accountType;
}
```

Implementation Notes: The class implements thread-safe balance updates using synchronized methods to prevent race conditions. The PIN verification uses hashed values for security, while maintaining immutability for critical account details.

Transaction

The Transaction interface establishes a common framework for handling all financial transactions within the ATM system. It defines essential methods for processing transactions, validating transaction data, and managing transaction types. This interface ensures consistency across different transaction types, facilitates easy integration of new transaction types, and maintains a clear separation between transaction logic and other system components.

```
// Core transaction interface for all ATM operations
// Defines basic transaction processing and validation
public interface Transaction {
    // Processes the transaction and returns success/failure
    // Main entry point for transaction execution
    boolean processTransaction();

    // Validates transaction data before processing
    // Ensures all required fields are valid
    boolean validate();

    // Gets the transaction type for logging and display
    // Used for transaction history and receipts
    TransactionType getTransactionType();
}
```

Implementation Notes: The interface defines three core methods that form the transaction lifecycle: processing, validation, and type identification. This minimal interface ensures flexibility while maintaining clear contracts for transaction implementations.

Withdrawal

The Withdrawal class implements cash withdrawal transactions in the ATM system, handling balance checks, cash dispensing, and transaction logging. It ensures secure and accurate processing of withdrawal requests while maintaining data consistency and providing clear transaction records.

```
// Implements cash withdrawal transactions
// Handles balance checks and cash dispensing
public class Withdrawal implements Transaction {
    private Account account;
    private BigDecimal amount;
    private CashDispenser cashDispenser;
    private TransactionLog transactionLog;
    private boolean isProcessed;

    // Constructor initializes withdrawal with required components
    // Sets up dependencies for transaction processing
    public Withdrawal(
        Account account,
        BigDecimal amount,
        CashDispenser cashDispenser,
        TransactionLog transactionLog) {
        this.account = account;
        this.amount = amount;
        this.cashDispenser = cashDispenser;
        this.transactionLog = transactionLog;
        this.isProcessed = false;
    }

    // Processes the withdrawal transaction
    // Checks balance and dispenses cash if sufficient
    // Returns true if successful, false otherwise
    @Override
    public boolean processTransaction() {
        if (!validate()) {
            return false;
        }

        // Check if sufficient balance available
        if (account.getBalance().compareTo(amount) < 0) {
            return false;
        }

        // Update balance
        account.updateBalance(amount.negate());

        // Dispense cash
        boolean dispensed = cashDispenser.dispense(amount);
        if (!dispensed) {
            // Rollback balance if dispensing failed
            account.updateBalance(amount);
            return false;
        }

        // Log transaction
        transactionLog.logTransaction("Withdrawal: " + amount);

        isProcessed = true;
        return true;
    }

    // Validates withdrawal amount
    // Checks if amount is positive
    @Override
    public boolean validate() {
        return amount.compareTo(BigDecimal.ZERO) > 0;
    }

    // Returns transaction type for logging
    @Override
    public TransactionType getTransactionType() {
        return TransactionType.WITHDRAWAL;
    }
}
```

Implementation Notes: The class implements atomic transactions with rollback capabilities, ensuring data consistency. It uses BigDecimal for precise monetary calculations and maintains transaction logs for auditing purposes.

Deposit

The Deposit class implements cash deposit transactions in the ATM system, handling envelope verification, amount validation, and balance updates. It ensures secure processing of deposits while maintaining clear transaction records.

```
// Implements cash deposit transactions
// Handles envelope verification and balance updates
public class Deposit implements Transaction {
    private Account account;
    private BigDecimal amount;
    private DepositSlot depositSlot;
    private TransactionLog transactionLog;
    private boolean isProcessed;

    // Constructor initializes deposit with required components
    // Sets up dependencies for transaction processing
    public Deposit(
        Account account,
        BigDecimal amount,
        DepositSlot depositSlot,
        TransactionLog transactionLog) {
        this.account = account;
        this.amount = amount;
        this.depositSlot = depositSlot;
        this.transactionLog = transactionLog;
        this.isProcessed = false;
    }

    // Processes the deposit transaction
    // Updates balance after envelope verification
    // Returns true if successful, false otherwise
    @Override
    public boolean processTransaction() {
        if (!validate()) {
            return false;
        }

        // Update balance
        account.updateBalance(amount);

        // Log transaction
        transactionLog.logTransaction("Deposit: " + amount);

        isProcessed = true;
        return true;
    }

    // Validates deposit amount
    // Checks if amount is positive
    @Override
    public boolean validate() {
        return amount.compareTo(BigDecimal.ZERO) > 0;
    }

    // Returns transaction type for logging
    @Override
    public TransactionType getTransactionType() {
        return TransactionType.DEPOSIT;
    }
}
```

Implementation Notes: The class implements secure deposit processing with envelope verification and maintains transaction logs for auditing. It uses BigDecimal for precise monetary calculations and ensures data consistency through atomic balance updates.

BalanceInquiry

The BalanceInquiry class implements balance checking functionality in the ATM system, providing users with their current account balance information. It maintains transaction records for auditing purposes and ensures secure access to balance information.

```
// Implements balance inquiry transactions
// Provides current account balance information
public class BalanceInquiry implements Transaction {
    private Account account;
    private TransactionLog transactionLog;
    private boolean isProcessed;

    // Constructor initializes balance inquiry with required components
    // Sets up dependencies for transaction processing
    public BalanceInquiry(
        Account account,
        TransactionLog transactionLog) {
        this.account = account;
        this.transactionLog = transactionLog;
        this.isProcessed = false;
    }

    // Processes the balance inquiry transaction
    // Logs the balance check
    // Returns true if successful, false otherwise
    @Override
    public boolean processTransaction() {
        if (!validate()) {
            return false;
        }

        // Log transaction
        transactionLog.logTransaction("Balance Inquiry");

        isProcessed = true;
        return true;
    }

    // Validates balance inquiry
    // Always returns true as no parameters need validation
    @Override
    public boolean validate() {
        return true;
    }

    // Returns transaction type for logging
    @Override
    public TransactionType getTransactionType() {
        return TransactionType.BALANCE_INQUIRY;
    }
}
```

Implementation Notes: The class implements secure balance inquiry processing with transaction logging for auditing. It uses simple validation since no parameters need verification, while maintaining proper transaction records.

TransferFunds

The TransferFunds class implements fund transfer between accounts in the ATM system, handling balance checks, account updates, and transaction logging. It ensures secure and atomic transfers while maintaining data consistency across both accounts.

```
// Implements fund transfer between accounts
// Handles balance checks and account updates
public class TransferFunds implements Transaction {
    private Account fromAccount;
    private Account toAccount;
    private BigDecimal amount;
    private TransactionLog transactionLog;
    private boolean isProcessed;

    // Constructor initializes transfer with required components
    // Sets up dependencies for transaction processing
    public TransferFunds(
        Account fromAccount,
        Account toAccount,
        BigDecimal amount,
        TransactionLog transactionLog) {
        this.fromAccount = fromAccount;
        this.toAccount = toAccount;
        this.amount = amount;
        this.transactionLog = transactionLog;
        this.isProcessed = false;
    }

    // Processes the transfer transaction
    // Updates both accounts if sufficient funds
    // Returns true if successful, false otherwise
    @Override
    public boolean processTransaction() {
        if (!validate()) {
            return false;
        }

        // Check if sufficient balance
        if (fromAccount.getBalance().compareTo(amount) < 0) {
            return false;
        }

        // Update balances
        fromAccount.updateBalance(amount.negate());
        toAccount.updateBalance(amount);

        // Log transaction
        transactionLog.logTransaction("Transfer: " + amount);

        isProcessed = true;
        return true;
    }

    // Validates transfer amount
    // Checks if amount is positive
    @Override
    public boolean validate() {
        return amount.compareTo(BigDecimal.ZERO) > 0;
    }

    // Returns transaction type for logging
    @Override
    public TransactionType getTransactionType() {
        return TransactionType.TRANSFER;
    }
}
```

Implementation Notes: The class implements atomic fund transfers with rollback capabilities, ensuring data consistency across both accounts. It uses BigDecimal for precise monetary calculations and maintains transaction logs for auditing purposes.

Card

The Card class represents a bank card in the ATM system, encapsulating card details, including the card number, expiration date, and associated account. It manages card validity and provides methods for card information access.

```
// Represents a bank card in the ATM system
// Stores card details and manages card validity
public class Card {
    private String cardNumber;
    private String expiryDate;
    private Account account;

    // Constructor initializes card with required components
    // Sets up card details and associated account
    public Card(
        String cardNumber,
        String expiryDate,
        Account account) {
        this.cardNumber = cardNumber;
        this.expiryDate = expiryDate;
        this.account = account;
    }

    // Gets the card number for identification
    // Used for card validation and account lookup
    public String getCardNumber() {
        return cardNumber;
    }

    // Gets the expiry date for validation
    // Used to check if card is expired
    public String getExpiryDate() {
        return expiryDate;
    }

    // Gets the associated account
    // Used for accessing account details and balance
    public Account getAccount() {
        return account;
    }
}
```

Implementation Notes: The class implements secure card data access with immutable properties for security. It maintains a clear relationship between cards and accounts while providing controlled access to card information.

CardReader

The CardReader class implements card reading functionality in the ATM system, handling card insertion, data extraction, and validation. It manages the interface between physical card data and the system while ensuring secure card processing.

```
// Implements card reading functionality
// Handles card insertion and data extraction
public class CardReader {
    // Reads card data when card is inserted
    // Returns null if reading fails
    public Card readCard() {
        // Implementation would read from hardware
        return null;
    }

    // Ejects the card from the reader
    // Used when session ends or transaction completes
    public void ejectCard() {
        // Implementation would control hardware
    }
}
```

Implementation Notes: The class implements secure card data extraction with error handling. It maintains a clear interface between hardware and software while ensuring proper card processing.

Keypad

The Keypad class implements input processing for PIN entry and transaction selections in the ATM system. It manages secure input collection and validation.

```
// Implements keypad input processing
// Handles PIN entry and transaction selections
public class Keypad {
    // Reads PIN entered by user
    // Returns null if reading fails
    public int readPin() {
        // Implementation would read from hardware
        return 0;
    }

    // Reads transaction selection from user
    // Returns null if reading fails
    public int readSelection() {
        // Implementation would read from hardware
        return 0;
    }
}
```

Implementation Notes: The class implements secure input processing with error handling. It maintains a clear interface between hardware and software while ensuring proper input validation.

CashDispenser

The CashDispenser class implements cash dispensing functionality in the ATM system, managing cash inventory and dispensing operations. It ensures accurate cash dispensing while maintaining inventory records.

```
// Implements cash dispensing functionality
// Manages cash inventory and dispensing operations
public class CashDispenser {
    // Dispenses specified amount of cash
    // Returns true if successful, false otherwise
    public boolean dispense(BigDecimal amount) {
        // Implementation would control hardware
        return true;
    }

    // Checks if sufficient cash available
    // Returns true if enough cash, false otherwise
    public boolean hasCash(BigDecimal amount) {
        // Implementation would check inventory
        return true;
    }
}
```

Implementation Notes: The class implements secure cash dispensing with inventory management. It maintains a clear interface between hardware and software while ensuring proper cash handling.

DepositSlot

The DepositSlot class implements deposit envelope processing in the ATM system, handling envelope reception and validation. It manages secure deposit collection while maintaining proper validation.

```
// Implements deposit envelope processing
// Handles envelope reception and validation
public class DepositSlot {
    // Receives deposit envelope and returns amount
    // Returns null if reception fails
    public BigDecimal receiveEnvelope() {
        // Implementation would control hardware
        return null;
    }
}
```

Implementation Notes: The class implements secure deposit processing with error handling. It maintains a clear interface between hardware and software while ensuring proper envelope validation.

ATMSystem

The ATMSystem class implements the core functionality of the ATM system, coordinating all components and managing state transitions. It maintains system state and provides methods for state management.

```
// Implements core ATM system functionality
// Manages state and component interactions
public class ATMSystem {
    // Current state of the system
    private ATMState currentState;

    // Card reader component
    private CardReader cardReader;

    // Keypad component
    private Keypad keypad;

    // Cash dispenser component
    private CashDispenser cashDispenser;

    // Deposit slot component
    private DepositSlot depositSlot;

    // Display component
    private Display display;

    // Constructor initializes system with components
    // Sets up initial state
    public ATMSystem(
        CardReader cardReader,
        Keypad keypad,
        CashDispenser cashDispenser,
        DepositSlot depositSlot,
        Display display) {
        this.cardReader = cardReader;
        this.keypad = keypad;
        this.cashDispenser = cashDispenser;
        this.depositSlot = depositSlot;
        this.display = display;
        this.currentState = new IdleState();
    }

    // Gets current state
    // Used for state management
    public ATMState getCurrentState() {
        return currentState;
    }

    // Sets new state
    // Used for state transitions
    public void setCurrentState(ATMState state) {
        this.currentState = state;
    }

    // Gets card reader component
    // Used for card operations
    public CardReader getCardReader() {
        return cardReader;
    }

    // Gets keypad component
    // Used for input operations
    public Keypad getKeypad() {
        return keypad;
    }

    // Gets cash dispenser component
    // Used for cash operations
    public CashDispenser getCashDispenser() {
        return cashDispenser;
    }

    // Gets deposit slot component
    // Used for deposit operations
    public DepositSlot getDepositSlot() {
        return depositSlot;
    }

    // Gets display component
    // Used for user interface
    public Display getDisplay() {
        return display;
    }
}
```

Implementation Notes: The class implements state management and component coordination. It maintains a clear interface between components while ensuring proper state transitions.

ATMState

The ATMState class implements the state management for the ATM system, handling state transitions and component interactions. It provides default implementations for state operations.

```
// Implements state management for ATM system
// Handles state transitions and component interactions
public class ATMState {
    // Handles card insertion event
    // Default implementation throws exception
    public void cardInsertedHandle(ATMSystem system) {
        throw new IllegalStateException("Invalid operation");
    }

    // Handles PIN entered event
    // Default implementation throws exception
    public void pinEnteredHandle(ATMSystem system, int pin) {
        throw new IllegalStateException("Invalid operation");
    }

    // Handles transaction selected event
    // Default implementation throws exception
    public void transactionSelectedHandle(ATMSystem system, int selection) {
        throw new IllegalStateException("Invalid operation");
    }

    // Handles withdrawal amount entered event
    // Default implementation throws exception
    public void withdrawalAmountEnteredHandle(ATMSystem system, int amount) {
        throw new IllegalStateException("Invalid operation");
    }

    // Handles cash dispensed event
    // Default implementation throws exception
    public void cashDispensedHandle(ATMSystem system, CashDispenser cashDispenser, Screen screen) {
        throw new IllegalStateException("Invalid operation");
    }
}
```

Implementation Notes: The class implements state management with default error handling. It provides a clear interface for state transitions while ensuring proper error handling.

IdleState

The IdleState class implements the initial state of the ATM system, handling card insertion and system initialization. It manages the transition to PinEntryState when a valid card is inserted.

```
// Implements initial state of ATM system
// Handles card insertion and system initialization
public class IdleState extends ATMState {
    // Handles card inserted event
    // Transitions to PinEntryState if card is valid
    @Override
    public void cardInsertedHandle(ATMSystem system, CardReader cardReader, Keypad keypad, Screen screen) {
        // Implementation would check card validity
        // Transition to PinEntryState if valid
        // Display error if invalid
    }
}
```

Implementation Notes: The class implements state transition with card validation. It maintains a clear interface between states while ensuring proper error handling.

PinEntryState

The PinEntryState class implements PIN verification in the ATM system, handling PIN entry and validation. It manages the transition to TransactionSelectionState when a valid PIN is entered.

```
// Implements PIN entry state
// Handles PIN verification and transition
public class PinEntryState extends ATMState {
    // Handles PIN entered event
    // Transitions to TransactionSelectionState if PIN is valid
    @Override
    public void pinEnteredHandle(ATMSystem system, int pin, Keypad keypad, Screen screen) {
        // Implementation would verify PIN
        // Transition to TransactionSelectionState if valid
        // Display error if invalid
    }
}
```

Implementation Notes: The class implements state transition with PIN validation. It maintains a clear interface between states while ensuring proper error handling.

TransactionSelectionState

The TransactionSelectionState class implements transaction selection in the ATM system, handling user choices and transition to appropriate transaction states.

```
// Implements transaction selection state
// Handles user choices and transition
public class TransactionSelectionState extends ATMState {
    // Handles transaction selected event
    // Transitions to appropriate state based on selection
    @Override
    public void transactionSelectedHandle(ATMSystem system, int selection, Keypad keypad, Screen screen) {
        // Implementation would handle selection
        // Transition to appropriate state
        // Display error if invalid
    }
}
```

Implementation Notes: The class implements state transition with selection validation. It maintains a clear interface between states while ensuring proper error handling.

WithdrawalProcessingState

The WithdrawalProcessingState class implements withdrawal processing in the ATM system, handling amount entry and transition to CashDispensingState.

```
// Implements withdrawal processing state
// Handles amount entry and transition
public class WithdrawalProcessingState extends ATMState {
    // Handles withdrawal amount entered event
    // Transitions to CashDispensingState if amount is valid
    @Override
    public void withdrawalAmountEnteredHandle(ATMSystem system, int amount, Keypad keypad, Screen screen) {
        // Implementation would process amount
        // Transition to CashDispensingState if valid
        // Display error if invalid
    }
}
```

Implementation Notes: The class implements state transition with amount validation. It maintains a clear interface between states while ensuring proper error handling.

CashDispensingState

The CashDispensingState class implements cash dispensing in the ATM system, handling dispensing and transition to IdleState.

```
// Implements cash dispensing state
// Handles dispensing and transition
public class CashDispensingState extends ATMState {
    // Handles cash dispensed event
    // Transitions to IdleState after dispensing
    @Override
    public void cashDispensedHandle(ATMSystem system, CashDispenser cashDispenser, Screen screen) {
        // Implementation would dispense cash
        // Transition to IdleState
        // Display error if failed
    }
}
```

Implementation Notes: The class implements state transition with dispensing validation. It maintains a clear interface between states while ensuring proper error handling.

```
public interface Transaction {
    TransactionType getType();

    boolean validateTransaction();

    void executeTransaction();
}

// Handles the withdrawal transaction process for removing funds from an account
public class WithdrawTransaction implements Transaction {
    Account account;
    BigDecimal amount;

    // Returns the transaction type as WITHDRAW
    @Override
    public TransactionType getType() {
        return TransactionType.WITHDRAW;
    }

    // Validates if the account has sufficient funds for withdrawal
    @Override
    public boolean validateTransaction() {
        assert account != null;
        return account.getBalance().compareTo(amount) > 0;
    }

    // Creates a new withdrawal transaction, throws exception if validation fails
    public WithdrawTransaction(Account account, BigDecimal amount) {
        if (!validateTransaction()) {
            throw new IllegalStateException(
                    "Cannot complete withdrawal: Insufficient funds in account");

        }
        this.account = account;
        this.amount = amount;
    }

    // Executes the withdrawal by subtracting the amount from account balance
    @Override
    public void executeTransaction() {
        account.updateBalanceWithTransaction(amount.negate());
    }
}

// Handles the deposit transaction process for adding funds to an account
public class DepositTransaction implements Transaction {
    final Account account;
    final BigDecimal amount;

    // Returns the transaction type as DEPOSIT
    @Override
    public TransactionType getType() {
        return TransactionType.DEPOSIT;
    }

    // Deposit transactions are always valid
    @Override
    public boolean validateTransaction() {
        return true;
    }

    public DepositTransaction(Account account, BigDecimal amount) {
        this.account = account;
        this.amount = amount;
    }

    // Executes the deposit by adding the amount to the account balance
    @Override
    public void executeTransaction() {
        account.updateBalanceWithTransaction(amount);
    }
}
```

The Transaction interface includes essential methods to manage transactions:

- getType(): Returns the transaction type (e.g., WITHDRAW or DEPOSIT).
- validateTransaction(): Validates the transaction's feasibility, such as checking funds availability for withdrawals.
- executeTransaction(): Executes transaction by updating the account balance.

ATMState

The ATMState abstract class forms the foundation of our ATM's state machine, ensuring that each stage of the user flow, from card insertion to transaction completion, follows a consistent process. Its seven concrete implementations manage distinct stages, enabling the ATMMachine to execute the user flow by transitioning between states as the user progresses through their session.

The ATMState interface includes essential methods to manage each stage:

- processCardInsertion(String cardNumber): Handles card insertion, used at the start of a session.
- processCardEjection(): Handles card ejection, which is used to end a session.
- processPinEntry(String pinNumber): Processes PIN entry for authentication.
- processWithdrawalRequest(): Initiates a withdrawal transaction.
- processDepositRequest(): Initiates a deposit transaction.
- processAmountEntry(BigDecimal amount): Processes the amount for withdrawals.
- processDepositCollection(BigDecimal amount): Processes the amount for deposits.

Below is the implementation for the ATMState interface, showing its default behavior, followed by implementations of IdleState and WithdrawAmountEntryState:

```
public class ATMState {
    // Displays an invalid action message on the ATM screen
    private static void renderDefaultAction(ATMMachine atmMachine) {
        atmMachine.getDisplay().showMessage("Invalid action, please try again.");
    }

    // Default implementation for card insertion
    public void processCardInsertion(ATMMachine atmMachine, String cardNumber) {
        renderDefaultAction(atmMachine);
    }

    // Default implementation for card ejection
    public void processCardEjection(ATMMachine atmMachine) {
        renderDefaultAction(atmMachine);
    }

    // Default implementation for PIN entry
    public void processPinEntry(ATMMachine atmMachine, String pin) {
        renderDefaultAction(atmMachine);
    }

    // Default implementation for withdrawal request
    public void processWithdrawalRequest(ATMMachine atmMachine) {
        renderDefaultAction(atmMachine);
    }

    // Default implementation for deposit request
    public void processDepositRequest(ATMMachine atmMachine) {
        renderDefaultAction(atmMachine);
    }

    // Default implementation for amount entry
    public void processAmountEntry(ATMMachine atmMachine, BigDecimal amount) {
        renderDefaultAction(atmMachine);
    }

    // Default implementation for deposit collection
    public void processDepositCollection(ATMMachine atmMachine, BigDecimal amount) {
        renderDefaultAction(atmMachine);
    }
}
```

The IdleState waits for the user to insert a card and start the session, supporting only card insertion. If the card is valid, it transitions to PinEntryState. Otherwise, it displays an error.

```
public class IdleState extends ATMState {
    /**
     * This method is called when a card is inserted into the ATM. This transitions the ATM to the
     * PinEntryState if the card is valid.
     */
    @Override
    public void processCardInsertion(ATMMachine atmMachine, String cardNumber) {
        if (atmMachine.getBankInterface().validateCard(cardNumber)) {
            atmMachine.getDisplay().showMessage("Please enter your PIN");
            atmMachine.transitionToState(new PinEntryState());
        } else {
            atmMachine.getDisplay().showMessage("Invalid card. Please try again.");
        }
    }
}
```

The EnterWithdrawalAmountState manages the step where the user enters a withdrawal amount, coordinating with the ATMMachine to execute the withdrawal via the Bank, using a WithdrawTransaction to validate funds and update the account balance.

```
public class WithdrawAmountEntryState extends ATMState {
    // Handles card ejection by cancelling transaction and returning to idle state
    @Override
    public void processCardEjection(ATMMachine atmMachine) {
        atmMachine.getDisplay().showMessage("Transaction cancelled, card ejected");
        atmMachine.transitionToState(new IdleState());
    }

    // Processes withdrawal request by checking balance and dispensing cash if sufficient funds
    @Override
    public void processAmountEntry(ATMMachine atmMachine, BigDecimal amount) {
        String cardNumber = atmMachine.getCardProcessor().getCardNumber();
        Account account = atmMachine.getBankInterface().getAccountByCard(cardNumber);
        boolean isSuccess = atmMachine.getBankInterface().withdrawFunds(account, amount);

        if (isSuccess) {
            atmMachine.getCashDispenser().dispenseCash(amount);
            atmMachine.getDisplay().showMessage("Please take your cash.");
        } else {
            atmMachine.getDisplay().showMessage("Insufficient funds, please try again.");
        }
        atmMachine.transitionToState(new TransactionSelectionState());
    }
}
```

For brevity, we have omitted the code for the PinEntryState, TransactionSelectionState, and DepositCollectionState classes. The complete code for all state classes is available in the accompanying materials of this book.

ATMMachine

The ATMMachine class acts as the central controller of the ATM system, applying the Facade pattern to offer a simplified interface to the underlying complexities. It manages the ATM's hardware components, delegates state-specific behavior to corresponding ATMState classes such as IdleState and DepositCollectionState, and utilizes the BankInterface to perform banking operations.

Here is the implementation of this class.

```
// Main ATM machine class that manages the state and hardware components of the ATM
public class ATMMachine {
    private ATMState state;

    private final CardProcessor cardProcessor;
    private final DepositBox depositBox;
    private final CashDispenser cashDispenser;
    private final Keypad keypad;
    private final Display display;

    private final Bank bank;

    // Initializes ATM with all required hardware components and bank interface
    public ATMMachine(
            Bank bank,
            CardProcessor cardProcessor,
            DepositBox depositBox,
            CashDispenser cashDispenser,
            Keypad keypad,
            Display display) {
        this.bank = bank;
        this.cardProcessor = cardProcessor;
        this.depositBox = depositBox;
        this.cashDispenser = cashDispenser;
        this.keypad = keypad;
        this.display = display;
        this.state = new IdleState();
    }

    // Forwards card insertion to current state for processing
    public void insertCard(String cardNumber) {
        state.processCardInsertion(this, cardNumber);
    }

    // Forwards card ejection to current state for processing
    public void ejectCard() {
        state.processCardEjection(this);
    }

    // Forwards PIN entry to current state for validation
    public void enterPin(String pin) {
        state.processPinEntry(this, pin);
    }

    // Forwards withdrawal request to current state for processing
    public void withdrawRequest() {
        state.processWithdrawalRequest(this);
    }

    // Forwards deposit request to current state for processing
    public void depositRequest() {
        state.processDepositRequest(this);
    }

    // Forwards amount entry to current state for processing
    public void enterAmount(BigDecimal amount) {
        state.processAmountEntry(this, amount);
    }

    // Forwards deposit collection to current state for processing
    public void collectDeposit(BigDecimal amount) {
        state.processDepositCollection(this, amount);
    }

    // Returns the display component for showing messages
    public Display getDisplay() {
        return display;
    }

    // Returns the cash dispenser component for handling withdrawals
    public CashDispenser getCashDispenser() {
        return cashDispenser;
    }

    // Returns the bank interface for account operations
    public BankInterface getBankInterface() {
        return bank;
    }

    // Returns the card processor component for handling card operations
    public CardProcessor getCardProcessor() {
        return cardProcessor;
    }

    // Returns the keypad component for user input
    public Keypad getKeypad() {
        return keypad;
    }

    // Updates the current state of the ATM
    public void transitionToState(ATMState nextState) {
        this.state = nextState;
    }

    // Returns the current state of the ATM
    public ATMState getCurrentState() {
        return state;
    }

    // Returns the deposit box component for handling deposits
    public DepositBox getDepositBox() {
        return depositBox;
    }
}
```

Hardware Management:

- The ATMMachine class manages all hardware components, including the card processor, cash dispenser, deposit box, keypad, and display, to enable user interactions.
- It provides access to these hardware components through getter methods like getCardProcessor and getDisplay for use by state classes.

State Management:

- The ATMMachine class maintains the current state of the ATM using a state field to track the active stage of the user flow.
- It provides state transition functionality through the setState method, allowing the ATM to move between stages like IdleState and PinEntryState.

Wrap Up

With the ATM system fully implemented, it's time to step back and consider what we've achieved. This chapter began by gathering requirements through a structured dialogue, then progressed to defining core objects like accounts and transactions, designing their class structure, and coding the essential components, including the state machine and hardware interactions.

The system's maintainability and extensibility are ensured by the clear division of responsibilities among the classes: Account and Bank manage account data and banking operations, Transaction handles financial operations, ATMState and its state classes (IdleState, PinEntryState, etc.) manage the user flow stages, hardware interfaces (Keypad, CardProcessor, etc.) handle user interactions, and ATMMachine orchestrates the entire flow as a facade. Our choices, such as using the state pattern with ATMState and separating hardware interactions into interfaces, improve modularity and allow future extensions, like adding new transaction types or hardware components.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor
Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information
Support
hi@bytebytego.com
Report a Bug
Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews
Ask AI
Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.